Coding and Configuration for Secure Systems in BSDs

John O'Keefe-Odom

Contents

Author

John O'Keefe-Odom received a Master of Liberal Arts (ALM) in Extension Studies in the field of Cybersecurity from Harvard Univerisity, Harvard University Extension School, in 2026. He received an AAS in Web Programming from Chattanooga State Community College in 2013. He received a BA in English: Writing from the University of Tennessee at Chattanooga in 2001. He holds various technical certifications in cybersecurity including: Certified Information Systems Security Professional (CISSP), Certified Secure Software Lifecycle Professional (CSSLP), Certified Cloud Security Professional (CCSP), SecurityX, PenTest+, CySA+, Security+, and Linux+. He has worked as a senior application security engineer and software engineer for various companies for over ten years.

Preface

Your preface text goes here...

Acknowledgements

Your acknowledgements text goes here...

With our chosen style of citations, we tried to provide references that would withstand common lab use and academic rigor. We provided in-chapter sections for convenient references to books, man pages, and websites. We also continued with inline footnoting for chapter endnotes; we especially noted various computer commands and technologies since they are fundamentally derivative works upon which we depend. This made for a lot of references and some duplicity. We have cited generously.

We decided to set a previous book in fonts by Erik Spiekermann after seeing "Helvetica" by Gary Hustwit (2007). We carried that through here. InfoOTDisp-Book is copyrighted by Ole Schaefer and Erik Spiekermann; it is published by FSI Fonts und Software GmbH. Eurostile Extended Regular and Eurostile W01 Regular are copyrighted 2006 and 2010 by URW Design and Development.

For typesetting and formatting, we chose LaTex. We repeatedly consulted various AI and Internet sources for coding the document which generated the paper. For bibliographic formatting, we frequently used MyBib. [1]

Introduction

Your introduction text goes here...

1 Understanding Risks to Physical Systems

We conducted a risk assessment to better understand the threat of criminal action against an in-house system. We examined both cybercrimes and physical crimes using data that was normalized to a per-capita rates of crimes. We drew from FBI Internet Crime Center (ICC) annual reports and Chattanooga Police open data records. The population samples were taken from the US Census Bureau reports.

1.1 Introduction

1.2 Methodology

1.3 Results

Text with[2] inline references. More text[3]

Endnotes

[1]Mybib. "MyBib Bibliography Generator." MyBib.Com, 13 July 2018, https://www.mybib.com. Accessed 12 July 2026.

[2]First note.

[3]Second note.

2 Amnesiac Systems in FreeBSD

In this technical note, we explore configurations of the FreeBSD operating system. With a goal of creating an amnesiac system that keeps the operating system separate from working storage drives, we experiment with various configurations. We demonstrate the planning of drive indexconfigurationconfiguration coding and the reasoning behind the choices. First, we code a system that uses the operating system on the storage drive discs. Then we evolve the plan into using a live cd for operating system use with drives that are connected to the immutable system in Random Access Memory (RAM) via configuration code. Using an amensiac system like this, we would be able to reset the operating system to a known immutable state with every startup. The drives could be disconnected and reattached to other systems for various purposes. Throughout, we cover specific commands for drive and operating system management and operation.

2.1 Introduction

2.1.1 Background and Content

2.1.2 Problem Statement

Attackers cannot destructively interact with systems without writing to them. Our goal is to keep the core services and operating system stable and immutable through physical security controls. This will allow the system to resist common attacks such as the encryption of files from ransomware and persistent account takeovers.

2.1.3 Project Objectives

Our system, hostname Blondie, set an example for amnesiac system implementation and design with FreeBSD. We hope that this example system will raise awareness and set patterns that will allow others to gain some of the same risk controls in their systems.

2.1.4 Scope of Work

Hypothesis: H1

An immutable operating system will be more resistant to destructive attack. Attackers cannot destructively interact with systems without writing to them.

Hypothesis: H2

An immutable operating system will promote system integrity. With ransomware as a dominant contemporary threat, integrity of the CIA triad rises to the forefront of defensive needs.

Hypothesis: H3

An immutable operating system will promote rapid recovery. With core components and configurations unchanged, an immutable system restarted after an attack will experience a more rapid recovery.

2.1.5 Variables

Non-Adversarial Threats

- Hardware limitations
- Coding and configuration errors
- Physical deficiencies
- Disasters

Adversarial Threats

- Ransomware
- Network attacks
- Physical attacks
- Non-malicious insider threats

2.1.6 Assumptions, Risks, and Limitations

Assumptions

- Sufficient time and control for power up, power down, and maintenance operations
- Direct control of server stack allows for optical disc use
- No migration of an existing commercial stack
- No commercial pressure for business performance or profit

Risks

- New procedures require rehearsal
- Complex research patterns may cloud known references
- Endeavor may be too complex to implement for some teams
- Unforseen compatability or integration problems

Limitations

- Increased complexity requires trained personnel
- Untaught patterns will require time to learn through experimentation
- No known defense against zero day vulnerabilities in the OS
- Size constraints on optical discs may limit immutable segment of system

2.1.7 Significance of the Project

2.2 Literature Review

2.2.1 Overview of Existing Solutions or Approaches

Immutability as a Constraining Concept

We wanted to better understand immutability as an idea. We came across an article (”Immutability as an Operational Constraint Rather Than a Security Feature”)[1] that pointed out that immutability can be a limiting factor. The author warned, ”By just making something immutable you haven’t necessarily made it safe; what you have effectively done is taken away the possibility of alteration and replaced it with a new operational reality: correction must happen through additional writes, and recovery must happen through reconciliation rather than repair”[2]

In this approach, the entire system was assumed to be an immutable system. This helped us to realize what a benefit it was to not make the entire system immutable. Our original intention was to have centralized, core, aspects of the operating system be immutable; other parts of the system could be changed within the rules and limits set by the OS.

Still, those changeable parts might have logging and monitoring activities, such as database snapshots set up in a way that allowed these cautions about immutability to contribute to our planning. Mallikarjun Vppalapati pointed out problems that could be had when records of on-system actions became immutable. Suppose a log or record was contaminated with malware or privacy information that needed to be forgotten under General Data Protection Regulation (GDPR). Administrators might need to undertake complicated edits to allow future auditors to bypass data that should not be considered. Vppalapati outlined a situation in which a simple programmatic error needed an elaborate and complicated fix because simple overwrites were not available.[3]

Vppalapati described three kinds of immutability: hard, soft, and operational. ”Hard immutability describes a state of being unchangeable where changes are so heavily protected by technical mechanisms and therefore practically infeasible the alteration process is therefore practically impossible.”[4] ”Soft immutability...can be reverted by using privileged credentials.”[5] ”Operational immutability implies the lack of changes without the use of technology”[6] In our proposed system, we would use hard immutability in the optical read-only recording of the core OS. We would use operational immutability as a control because we were applying an optical disc that required physical access to the server for alterations. For example, to change the operating system burned to optical disc, an attacker or operator would need to build a new disc and physically insert it into the system, and then follow our proscribed startup sequence to get the Operating System (OS) into RAM. We would also use soft immutability, in the writing of data to files in attached storage that was not automatically loaded by the system on boot.

Immutability Examples in Blockchains

If there's a technology that has a reputation for immutability, it is the blockchain. Simply put, the blockchain is a chain of cryptographic records; the starting point for the math used to encrypt each block is based on the previous record; the cummulative effect of the sequence of encryption is that if one record is corrupted or modified, then the break in the logic for decrypting subsequent blocks will reveal the change. So long as the records in the blockchain are accessible, users can assume that the previous entries have been unmodified. Blockchain has other characteristics; its records are usually distributed over networks and internets; but, the mathematical sequence of encryption and decryption based on preceding blocks is the source of its logical strength as an immutable record.

Blockchains, being immutable, can run into conflict with one of the more famous requirements for mutability of our time: GDPR's Right to be Forgotten (RtbF). [7][8] One of the components of GDPR, Recital 65, says that a person who has their data recorded has the right to request a modification or deletion of that data. There are some exceptions and provisions for children who get their data recorded, but at its core Recital 65 provides every person in the EU with the legal right to have their records modified. [9] Recital 66 says that a data controller has to inform processors to erase links to or copies of personal data. This way a request to be forgotten can't just stop with one entity or be restricted to entities that the requestor discovers on their own. [10] In these laws, we don't see exceptions for immutable systems built-in. On the contrary, by being left out altogether the responsibility for dealing with these legal conflicts seems to go back on the immutable system owner. If a system were to operate that was truly immutable, then if that system contained personal data that was to be erased upon a given request: it might require the destruction of the whole system to bring the operator into legal compliance with GDPR. This might not be as catastrophic as it sounds; perhaps an operator could take a system offline, delete a record, and then completely regenerate the records with the required deletions omitted. However, multiple such mutations could lead to complexities in system maintenance with multiple required regenerations. The problem could get further complicated if the system had subsequent sequential modifications. Altogether, system operators would be cautioned not to store personal data in immutable file structures. Perhaps we might be further cautioned to keep immutable only what was operationally necessary as a safeguard against future laws or requirements. As we work through the implementation of our immutable system, we shall see that sometimes the constraints can bring us structure and sometimes the limitations are innovation accellerators.

Blockchains offer an example of Vppalapati's hard immutability and operational immutability. The European Union (EU)'s GDPR counters with an example of soft immutability requirements for certain kinds of data. We can conclude that most personal data would have to be kept off of the optical discs (and out of any network distributed blockchains). While most Personally Identifiable Information (PII) would not seem to be a part of an operating system, later we'll see that a user's full name is requested by the system as a normal part of user account setup. We could have a small conflict if we're not careful.

Immutability Examples in Database Administration

2.2.2 Comparisons of Technologies or Frameworks

... compare with diagrams

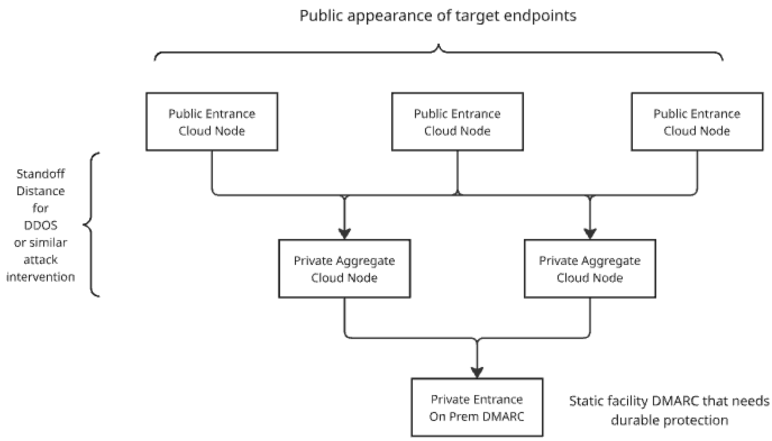

Fig. 2.1: Your caption

2.2.3 Gaps in the Literature

2.3 Methodology

2.3.1 Technical Research Questions

2.3.2 System Architecture and Design

2.3.3 Implementation Plan

We're starting with a Dell T610 Server. Made in 2011, this machine is about ten years old. We purchased this at a government e-waste auction. She needed some cleanup. We started on the front lawn, blowing out the dust with a leafblower.

Most of the time servers from these auctions will have minimal RAM. Usually the drives and any good accessories will be stripped out. Anytime we get a hard drive, from any source, one of the first actions we take is to erase the drives to United States (US) National Institute of Standards and Technology (NIST) 800-88 Purge[11] with dedicated devices. That kind of erasing can be done with `dd` in Kali or FreeBSD; but, the drive erasers usually have a verification feature that's helpful.

Hardware Configuration

This is my favorite computer. I've named her "Blondie" after Debbie Harry's New Wave band.[12]

Now, she's configured with:

- A DVD RW drive
- A LTO III[13] tape backup unit
- In her 8 drive bay, we have:
  - Two VelociRaptor[14] 80GB 10000 rpm drives.
    - ⋆ We'll use these in a Mirror configuration to hold the main OS.
  - Two refurb 120GB Solid State Drive (SSD)s
    - ⋆ We'll use this in a Mirror as ZFS log drives.
    - ⋆ They'll help to make writes faster.
  - Four 500GB hard drives
    - ⋆ We'll use this as a RAIDZ-2 array to hold the jails.
    - ⋆ We have some smaller Serial Attached SCSI (SAS) drives.
    - ⋆ We have some Serial AT Attachment (SATA) drives.
- We replaced the one CPU with two Xeon 5550[15] quad cores and added a cooling tower.
- We added more RAM. Currently, we're at 196GB.
- We added some more Network Interface Card (NIC)s.
  - We have four for the jails.
  - We have two for the OS.
  - We'll leave the two on the motherboard mostly unused, except for our iDRAC connection.
    - ⋆ iDRAC is a proprietary baseboard controller.
    - ⋆ We could use it to power cycle the machine.
    - ⋆ There's a web interface that let's us know about motherboard-related facts.
    - ⋆ We cannot give a GELI password through the console in our current setup.
- The server has two 500W power supplies.

Because the dual power supplies draw about 500 Watts apiece, this server gets it own circuit.

The drives we're using are older, but we got enough copies of the drives to be able to support backups. When making physical backups of the drives, it can be helpful to have other volumes of the same model. So, instead of getting the best available, we planned for the best we could afford at quantity.

The server does not have:

- Any type of graphics card or Graphics Processing Unit (GPU)
- Sound cards or microphone hardware
- Wireless networking or Bluetooth.

Intentions: Paving the Road to Jails

We'll use the box for development. It'll be our main development server. We'll use it with progRAMs that we compile on a nearby poudriere package building box. In those package builds, we have segregated our builds into a pattern of jails that mimic the types of jails that will be needed on most of our computers.

So, there is a jail for functional groups like: development tools, offensive and defensive security programs, browsers, desktop software, and so on. Poudriere jails can be filled with whatever sets of programs we like. For successful builds, we find its best to keep the jails small.

There will be three main groups of drives:

- an OS Mirror pair
- a ZFS log Mirror pair
- a ZFS RAIDZ-2 kit of four drives.

The OS pair will hold ideas like:

- jail configuration
- main host networking and Virtual Network (VNET)
- host-based user accounts and configuration.

The ZFS kits will hold disconnected jails. We'll set up a system of hierarchical jails that provide us with a development environment made up of modular jails that will be the children of an overarching parent. These jails will communicate with each other and the parent through some custom VNET. We'll do all of our jail scripting, controls, and configuration manually. Likewise, the base ZFS setup will be done with simple scripts.

Preparing the Baseboard

The T610 has a built-in RAID card that can't be totally bypassed. This means that since we'll want ZFS to control the drives as much as possible, we'll need to tell the hardware RAID to buzz off. We won't want hardware RAID to participate in the process. We'll just use the software RAID through ZFS.

We can't just unplug the RAID card. We tried that, and it leads to malfunctions on this machine. So, as a compromise, what we'll tell the machine is that each drive is its own VDEV and it's RAID-0. We'll end up with as many VDEVs as we have physical drives.

To prepare for a later step in this process, we'll note the serial number of each drive we load into position, its model number, and its capacity. We'll need to know that information, and we won't want to be hassled by having to power cycle the machine in the middle of the setup process just to check on those details.

Later on, we'll set VDEVs with ZFS; but, from this point on, we'll basically ignore the RAID card's description of these devices as VDEVs.

With the baseboard prepared, and the BIOS set to allow us to boot from disc, we'll get started with a plain install of FreeBSD to the first two drives in a Mirror configuration.

Incremental Advancement to Amnesiac System

With all of the manual coding and configuration needed to set up an amnesiac system like the one we desired, it was a big change from the normal BSD installs we had done for several years. Our first pass at building an amnesiac system decayed into manually scripting a regular operating system installation that had most of the features we desired in the amnesiac system. Both the on-disc OS system and the amnesiac system are detailed below.

Identifying and Allocating ZFS Drive Qualities

We need to be able to detect the drives and partitions that we want to work with using geom, gpart, and zfs commands. If, for some reason, you can't find those drives, then you need to address that. While working with this particular computer, I've found that the leading cause of failing to detect a drive goes back to the VDEV assignment in the hardware RAID. If the drive were pulled, this particular RAID card will need to have those configuration changes addressed. Often, this means rebooting the machine. It could be done through the iDRAC baseboard controller in some models; but the one we have here is not configured to do those tasks.

What are we going to want the ZFS kit to do? How are we going to use the drives? How can we wrestle with this new type of configuration?

After reading over FreeBSD Mastery: ZFS by Alan Jude and Michael Lucas[43], we came to a few conclusions. We chose to use RAIDZ-2 with Log drives because of the number of drives we have[44]. Some of the topics covered in that book that we'll want to apply include ideas in Table 2.1. In our first iterations we decided to install the OS on the hard drives in a pattern that looks like a conventional installation. Once the drive partitioning and utilitization commands were worked out, we then started over and undertook the amnesiac version with the OS on a live disc that runs with attached drives.

Our goal for the system will be to split up the drives into three main groups, as described in Tables 2.2, 2.3, and 2.4. On each drive, we'll also allow:

- 15% of the disk's own capacity as "Free Space" by allocating a blank partition
- Swap by 100 | 50 | 33 | 25 percent of RAM (with a 25% minimum for all disks) or similar
- Remainder as one large ZFS main partition.

We decided we would want those qualities in an amnesiac system, with the exception of running the OS from a live disc.

For the OS-On-Disk system, this should bring us:

- Enough room for the main OS
- A 4 disk RAIDZ-2 array of almost 1.49 TB of actual storage
- Fast swap and copy-on-write
- With each disk capable of expansion or change.

The unused space on the drives (a blank partition of 15%) may look like we're grossly under-utilizing the disks. We're planning for future emergencies. Do we need more

Table 2.1: ZFS Concepts and Commands

Topic	Concept	Command phrase example
Disk labels	Use the label to associate the logical with the physical[16]	`[Host]-[PositionNumber 0-7]-[Serial_Number]`[17]
ZFS Compression	Faster performance: less space[18]	`zfs set compress=gzip-6 [DIRECTORY]`[19]
ZFS zpool comment	Make a note on the pool[20]	`zfs set comment="-"`[21]
quota[22]	Limit growth[23]	–
reservation[24]	Set aside space for directory and its children[25]	–
refreservation[26]	Set aside space but don't count snapshots and child datasets[27]	–
canmount[28]	Control mounting[29]	`zfs set canmount=off [DIRECTORY]`[30]
mountpoint[31]	Specify where to mount[32]	`zfs set mountpoint=/SOME [DIRECTORY]`[33]
readonly[34]	Control readonly[35]	`zfs set readonly=off [DIRECTORY]`[36]
exec[37]	Control if it can execute[38]	`zfs set exec=off [DIRECTORY]`[39]
copies[40]	Make more than one copy[41]	`zfs set copies=2 [DIRECTORY]`[42]

Table 2.2: ZFS Main OS Use Hard Disk: Initial Estimates

	Estimate for 80 GB HDD
Blank: 15% ”Free Space”	12 GB
Swap: Under-allocate, deliberately	10 GB
ZFS Main Partition: Remainder	58 GB

Table 2.3: ZFS Jail Use Hard Disk: Initial Estimates

	Estimate for 500 GB HDD
Blank: 15% ”Free Space”	75 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Main Partition: Remainder	375 GB

swap? Maybe we can add it in. Do we need to expand or resize the disk? Maybe we can delete that blank ”Free Space” partition and use that. By not trying to use everything, we're giving ourselves some room to solve future problems.

What We Can Learn From Installing an OS

Later on, we'll install the other drives wtih gpart and zfs commands. This first pair, through, we could do with bsdinstall[45]. If we choose that, though, then we must commit the whole disk to the process. That would be how we would normally stand up a disk drive. In this case, we'll conduct a manual process that mimics the main steps of bsdinstall[46] to show that the installation can be done manually. Part of these same kinds of steps are critical tasks when administrating and installing jails.

Basic Operating system Installation Goals

We're going to do an early phase, basic installation of the OS. At this point, we'll really only set up a pair of users and some plain vanilla settings for the OS. We want to be able to:

- to establish our ZFS kit
- to configure our main jail relationships
- to be able to check our networking with other hosts

Table 2.4: ZFS SLOG Log Use SSD: Initial Estimates

	Estimate for 120 GB SSD
Blank: 15% ”Free Space”	18 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Log Partition: Remainder	52 GB

- to set the basic operating system to call packages from our package server.

We can come back later and make a more sophisticated environment. At this stage, we will just lay the foundation for that work. If we don't get the ZFS kit stood up, then our more complex work with jails won't have an environment in which to thrive.

Main Steps for FreeBSD Installation from bsdinstall

[47]

We can see that `bsdinstall` [48] is broken up into about a dozen main segments. They are:

- Keyboard mapping
- Hostname
- Networking configuration
- Wireless networking
- Partitioning
- Mounting temporary root
- Fetch the OS
- Extract the OS
- Set a root password
- Set the time
- Add users
- Enable some services
- Copy in the config [2].

We're not going to do all of these. We'll start with partitioning. We won't do any wireless networking or set a root password. We're going to skip the wireless networking for now because the hardware doesn't have that capability. In the newer versions of bsdinstall, there are some protective measures not listed above. We won't tackle those just yet.

No Root Password

We won't set a root password because we will expect to always Secure Shell (SSH) into the root account with a key. By not setting a root password, we will create a situation where a key is always required. This can be dangerous in that it can create a single point of failure for the system.

Another risk is that we will require the ability to SSH in as root. Later, as we establish software firewalls on the host, we will want to limit what addresses can be used to access this root with SSH. It's a little dangerous, but it's also a little different.

We have some risk controls that will be in place. First, we often store our keys in a special collection of USB drives. Next, we will have 2FA on the SSH that will require a TOTP (Time-based One Time Password). Finally, we will limit what's done in this drive. Since most of our programs and work will be done in the jails, there is a limited amount of influence that this part of the system will have. It's there to start the box and spin up the jails. If we ever have to totally destroy this installation and hook up our jails to another system, that is possible.

What do we gain from no root password? An extra measure of security. If there is no password, then password enumeration attempts should all fail. This not only goes for root, but it goes for other users. Many of the automated login attacks we've seen have been simple dictionary attacks; these brute force exercises rarely attempt to account for the types of login procedures that we expect to put in place.

Not having a root password may also truncate or stop our ability to use commands like `su` or `sudo`. As we're trying to stop privilege escalation, we'll also be stopping legitimate means of priv-esc. It could be inconvenient, but we've used this technique before.

Also, there is another risk: leaving the needed key lying around. For example, if we give the key to a user account operator, like ourselves, and then we import copies of the key to the box for easy use: well, then those keys could be used and re-used locally. This would be the equivalent for storing a plain-text password on the box. They keys are ultimately simple text files with decorated, long, strings of hashes. It's the software that goes with the keys, combined with the configuration of the progRAMs for logging in to a service, that makes them special. Switching from passwords to keys is not a perfect answer; but, it's the answer we'll use this time.

If someone wishes to put a root password in place, then that's a procedure that will be easy to implement. For now, though, there will be no root password. And that means we have to build the keys we will use to log in.

Be Able to Build the Keys

We can build our keys using the live disc's shell. and a thumbdrive. The main command to build the keys is:

`ssh-keygen -b 1024 -t rsa`[49]

Copy the files output to a thumbdrive and keep them handy for mounting.

Partition, Mount, and Use a Thumbdrive from Live Disc's Shell

Let's take some time to go over some detailed instructions on how to build those keys and get them onto a thumbdrive. The live disc's shell mostly uses a read-only file system. So, some people may not be familiar with how we are going to get the keys made and saved anywhere that will persist.

Start the Box and the Live Disk's Shell

With the hardware booted to the LiveCD (installation disc), we select " Shell" from the three main choices in the first menu. This drops us into the shell. If we try to save a file of any kind, we'll probably get an error message back that says we're in a read-only file system. Our main set of directions for this is in the FreeBSD Handbook. We look carefully at the sections for USB storage and "Adding Disks"

Be Able to Plug in the USB Thumbdrive and Find Its Logical Address

`camcontrol devlist`[50]

This should show the USB drive. If the list is long, we can pipe it to more and slowly advance through the list. In our case, the USB was at da0.

The listing should also show any other storage device that's plugged in to the machine. So, all of those hard drives loaded into the bays should appear. If they don't, then take it as a warning sign that we're having a problem with the RAID card's physical drive VDEVs. On the T610, we were seeing drives in positions 0 to 7 in addition to the USB thumbdrive we're working on now.

Be Able to Partition the USB

Since our USB has not been prepared for use with anything, we have to partition it. We can pretty much use the default commands fro 17.2 "Adding Disks" to set up the drive. Here, we create a simple UFS file system and mount it to a directory that we'll put in /tmp. If our USB is already partitioned, we would want to skip this part.

```
gpart create -s GPT da0[51]
gpart add -t freebsd-ufs -a 1M da0[52]
newfs -U /dev/da0p1[53]
```

Mount the USB to the /tmp Directory:

```
mkdir /tmp/someDir[54]
mount /dev/da0p1 /tmp/someDir[55]
```

Be Able to Write to the USB

We could now write a simple text file onto /tmp/someDir, unmount the drive, and then re-mount it to check our work. That would go like:

```
cd /tmp[56]
umount /tmp/someDir[57]
rm -r /tmp/someDir[58]
```

Unplug the drive. Plug it in another slot. Use camcontrol devlist to find it again. It'll probably get the same address, like "da0".

```
mkdir /tmp/otherDir[59]
mount /dev/da0p1 /tmp/otherDir[60]
ls /tmp/otherDir[61]
```

Be Able to Put Some Key Pairs on the USB

Using these same principles, we can mount our USB thumbdrive, generate SSH keys, and save them there for later.

```
mkdir /tmp/otherDir/keys[62]
ssh-keygen -b 1024 -t rsa[63]
```

During the ssh-keygen dialog, simply direct the progRAM to save the keys to `/tmp/otherDir/keys`. Be sure to record your passphrase somewhere.

Be Able to Use Commands to Build Out the Drives

We'd like to show a quick overview of some of the drive-related commands. These will be applied soon.

To list what devices are detected by the OS, we can use:

```
gpart show[64]
```

We will want to note the serial number of each drive, by position. We then note which drives we want to use for the main body of the RAIDZ-2 kit. All of this will become one pool of ZFS VDEVs. Then we'll add in the log mirrors. So, in the sequence of 0-7, numbering the drives, we should expect to skip those two for now.

Make the notes if you haven't already because we're going to list all of the drives for the main pool in one pop. This task may have been done before we set up the physical VDEVs for the hardware RAID.

We're going to choose to put a swap partition on the SSDs. We can add one onto the other disks, but a maximum of 4 swap partitions is recommended in the FreeBSD Handbook. Later on, we will be able to select swap partitions by mounting them and using commands like `swapon swapoff`.

To put the ZFS partition on the remaining disks:

```
gpart create -s SCHEME DISK_DEVICE[65]

gpart create -s gpt da4[66]
```

To add the main ZFS partition:

```
gpart add -a 1m -l ZFS_NUMBER \
-t freebsd-zfs DISK_DEVICE[67]

gpart add -a 1m -l ZFS-BLONDIE-4-WD123465789 \
-t freebsd-zfs da4[68]
```

To add the partition's swap:

```
gpart add -a 1m -slg -l SWAP_NUMBER \
-t freebsd-swap DISK_DEVICE[69]

gpart add -a 1m -slg -l SWAP-BLONDIE-4-WD123465789 \
-t freebsd-swap da4[70]
```

To add a logging partition on the SLOG SSDs:

```
gpart add -a 1m -l ZLOG_NUMBER \
-t freebsd-zfs DISK_DEVICE[71]

gpart add -a 1m -l ZLOG-BLONDIE-2-WD123465789 \
-t freebsd-zfs da2[72]
```

With each disk partitioned, we can add them to the pool for our RAIDZ-2 kit.

```
zpool create RAID POOL_NAME DISK_LABEL/ZFS_PARTITION_LABEL
...next disk/partition...[73]


zpool create BLONDIE-MAIN raidz2 \[74]
BLONDIE-4-WD12345/ZFS-BLONDIE-4-WD12345 \
BLONDIE-5-WD12345/ZFS-BLONDIE-5-WD12345 \
BLONDIE-6-WD12345/ZFS-BLONDIE-6-WD12345 \
BLONDIE-7-WD12345/ZFS-BLONDIE-7-WD12345
```

We can add on the mirrors for logging:

```
zpool add BLONDIE-MAIN log mirror \[75]
BLONDIE-2-WD12345/ZLOG-BLONDIE-2-WD12345 \
BLONDIE-3-WD12345/ZLOG-BLONDIE-3-WD12345
```

Once we get to the point that we have an operating system on disks, and disks ready for use with logs and jails, then we're at the end of this phase of system administration.

Goals for Amnesiac Systems

We chose at this point to do some experimental installs. For clarity, notes on those are not presented here. We did change our goals for the system. We recognized that we wanted:

- Use a LiveCD for an immutable OS
- Encrypt disc partitions instead of the whole disc

Table 2.5: FreeBSD Swap Disk – Estimate of 500 GB HDD

GEOM Disk List Mediasize	465 GB
Blank: 15% ”Free Space”	70 GB
Swap	395 GB

Table 2.6: ZFS Cache (Read Cache) Disk – Estimate of 80 GB HDD

GEOM Disk List Mediasize	74 GB
Blank: 15% ”Free Space”	11 GB
Swap	63 GB

- Ensure we save keys and metadata to promote disc recovery
- Prepare commands in advance to reduce complexity.

From our testing, we recognizes that a Live CD offered the best answer for an immutable system. It does come at the price of some instability. The ”El Torrito” discs have a limited set of services. Adjusting the configuration of the operating system means drafting, burning, and testing a new disc. The ” shell” instance on a FreeBSD installation disc is reportedly less stable; but, we did not know why or how it was less stable than a normal operating system. We worked through our disc commands and kept careful notes. Adding the physical serial number of the disc alone increased the complexity of manually typing in each command. We discovered the need to encrypt the discs by partitions instead of the whole disc because there was a visibility problem during the execution of discovering and attaching the discs. There was a poorly documented detail about protecting the disc metadata and keys. The keys are provided during the setup dialog; but, these highly important items will disappear after the setup process closes. This means it is important that an operator recognize the need to intercept and to record these keys. Without them, a later recovery of a disc is impossible. Since one typo would derail our experimental setups, we had to keep a second support computer on hand to record notes and consult references.

As our experimentation concluded, we had learned enough to stand a reasonable chance to setup an amnesiac machine. Its features include:

- An immutable OS
- Encrypted disc partitions
- Swap and logging for ZFS disc features
- Metadata and keys saved to a separate USB drive
- A disc for holding FreeBSD jails (containerization)
- Enough room to expand, to update, and to recover.

Table 2.7: ZFS SLOG Log Mirror (Write Cache) Disk – Initial Estimates

	Estimate for 120 GB SSD
GEOM Disk List Mediasize	465 GB
Blank: 15% ”Free Space”	18 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Log Partition: Remainder	52 GB

Table 2.8: ZFS Jail Use Hard Disk – Estimate of 500 GB HDD

	Estimate for 500 GB HDD
GEOM Disk List Mediasize	465 GB
Blank: 15% ”Free Space”	75 GB
Swap: 25% Physical RAM (196 GB max)	50 GB
ZFS Main Partition: Remainder	375 GB

Configure for SWAP Disk, ZFS Cache, ZFS SLOG Mirror, and ZFS RAIDz2

Changed config to keep the RAIDZ2 for main work. I was having trouble with the SSDs; so, I removed them and replaced them with two 80GB Western Digital Velociraptors. I added another WD Velociraptor for cache. Then we added a Western Digital Blue 500GB for a swap disk. That sounds like a lot of swap, but we hope to upgrade to near 196GB of RAM this year. So, a little more than twice that would be 400GB of swap. The next disk size on hand was 500GB.

We had some mis-aligned partitions with RAIDZ2 work for Blondie. We were using the full 500GB on the label as our actual amount of media to work with. Since less is available, we will have to set those up again.

We used gpart delete to delete each partition. Then we used gpart destroy to delete the partition table for the whole disk. In the end, gpart showed us only the CD.

Startup with **bootonly.iso**

For this run, we'll begin by using the boot-only disk. We won't plan to ever boot from these hard drives. We'll just use the shell on the CD. As with the others, we'll mount a USB to store metadata information. We'll partition and encrypt the disks. In the following examples, we provide the actual serial numbers for the hard drives that were used. This example was derived from notes we maintained during experimentation; serial numbers in other instances will vary, of course.

Mount the USB to Save the Metadata Backup File

```
mkdir /tmp/usb3[76]
mount /dev/da0p1 /tmp/usb3[77]
```

Create Swap Disk in Bay 0

```
gpart create -s GPT mfid0[78]
gpart add -a 5K -s 395G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-WCC2ERZ56512 mfid0[79]
```

Create ZFS Cache in Bay 1

```
gpart create -s GPT mfid1[80]

gpart add -a 5K -s 63G -t freebsd-zfs \
-l ZFS-BLONDIE-1-CACHE-WXL1E40C1352 mfid1[81]
```

Create ZFS SLOG in Bays 2 and 3

```
gpart create -s GPT mfid2[82]

gpart add -a 5K -s 63G -t freebsd-zfs \
-l ZFS-BLONDIE-2-SLOG-WXL1E4067503 mfid2[83]

gpart create -s GPT mfid3[84]

gpart add -a 5K -s 63G -t freebsd-zfs \
-l ZFS-BLONDIE-3-SLOG-WXC0CA9V8431 mfid3[85]
```

Create ZFS Work Disks for Jails

Bay 4

```
gpart create -s GPT mfid4[86]

gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-4-SWAP-WCC2EE134983 mfid4[87]

gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-4-FILES-WCC2EE134983 mfid4[88]
```

Bay 5

```
gpart create -s GPT mfid5[89]

gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-5-SWAP-WMC2E3914420 mfid5[90]
```

```
gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-5-FILES-WMC2E3914420 mfid5
```
[91]

Bay 6

```
gpart create -s GPT mfid6
```
[92]

```
gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-6-SWAP-WCC2ETT39577 mfid6
```
[93]

```
gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-6-FILES-WCC2ETT39577 mfid6
```
[94]

Bay 7

```
gpart create -s GPT mfid7
```
[95]

```
gpart add -a 5K -s 50G -t freebsd-swap \
-l ZFS-BLONDIE-7-SWAP-WCAYU2878968 mfid7
```
[96]

```
gpart add -s 345G -t freebsd-zfs \
-l ZFS-BLONDIE-7-FILES-WCAYU2878968 mfid7
```
[97]

We should note that “Bay 7” is an observation of the physical hardware. The device (“mfid7”) could have whatever number assigned at the end that the machine gave it. If the hardware RAID had a problem with recognizing a device, the number might not correlate to what bay it is in. When powering up, check your geometry.

Encrypt those partitions

During this phase, we'll encrypt the partitions. That means that we should have some recordkeeping materials on hand. Use a password manager or a notebook to carefully record the password that is about to be typed in. With each one completed, the machine will tell us that it has saved a file of backup metadata. We will want to copy that metadata file to the thumbdrive. If the encryption gets jammed up, or if there is some type of problem, then the metadata file will be used as part of the recovery process. Without a metadata file, recovering an encrypted partition can be difficult or impossible.

If you lose the metadata file, but still have routine access to the geli encryption because you know the password and the machine is functioning alright, then you can export another copy of the metadata. This means that anyone with root access to the machine, who can use geli commands, would also be able to export such a file. Keep this in mind if you need to secure your machine.

When we give the subcommand to “init,” the machine will ask us to give the new password twice. This is part of the password setting process. When we give the subcommand to “attach,” the machine will ask us to give the password as part of the normal attachment process. Type carefully, and look at the records carefully, because this is the main chance to make sure the password notes are right.

Commands to Encrypt the Partitions

```
geli init -l 256 mfid0p1[98]
geli attach mfid0p1[99]

geli init -l 256 mfid1p1[100]
geli attach mfid1p1[101]

geli init -l 256 mfid2p1[102]
geli attach mfid2p1[103]

geli init -l 256 mfid3p1[104]
geli attach mfid3p1[105]

geli init -l 256 mfid4p1[106]
geli attach mfid4p1[107]

geli init -l 256 mfid4p2[108]
geli attach mfid4p2[109]

geli init -l 256 mfid5p1[110]
geli attach mfid5p1[111]

geli init -l 256 mfid5p2[112]
geli attach mfid5p2[113]

geli init -l 256 mfid6p1[114]
geli attach mfid6p1[115]

geli init -l 256 mfid6p2[116]
geli attach mfid6p2[117]

geli init -l 256 mfid7p1[118]
geli attach mfid7p1[119]

geli init -l 256 mfid7p2[120]
geli attach mfid7p2[121]
```

To Preserve the Metadata Backup Files

Let us reiterate: preserving these backups at the time of disk creation is essential for later recoveries. If these files are not saved, then a powerdown of the system will cause them to be lost. They're automatically stored in an ephemeral directory. Conserve the ability to recover a disk later by recording these metadata backup files to a safe location.

Each metadata backup file was probably created with a name that does not look like the drive's label. Instead, the machine assigns a backup file name based on geometry. Since we can have a hardware configuration change that could mess up that pattern, we will want to save our backup files with names that look more like the disk labels. That way, if the drives move, then we can better discover which metadata file goes with it.

```
cp /var/backups/mfid0p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-0-SWAP-WCC2ERZ56512.eli[122]

cp /var/backups/mfid1p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-1-CACHE-WXL1E40C1352.eli[123]

cp /var/backups/mfid2p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-2-SLOG-WXL1E4067503.eli[124]

cp /var/backups/mfid3p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-3-SLOG-WXC0CA9V8431.eli[125]

cp /var/backups/mfid4p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-4-SWAP-WCC2EE134983.eli[126]

cp /var/backups/mfid4p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-4-FILES-WCC2EE134983.eli[127]

cp /var/backups/mfid5p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-5-SWAP-WMC2E3914420.eli[128]

cp /var/backups/mfid5p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-5-FILES-WMC2E3914420.eli[129]

cp /var/backups/mfid6p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-6-SWAP-WCC2ETT39577.eli[130]

cp /var/backups/mfid6p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-6-FILES-WCC2ETT39577.eli[131]

cp /var/backups/mfid7p1.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-7-SWAP-WCAYU2878968.eli[132]
```

```
cp /var/backups/mfid7p2.eli /tmp/usb3/backups
_27SEP2020/ZFS-BLONDIE-7-FILES-WCAYU2878968.eli
```
[133]

File Name Check: Editing Labels

We got through all that, and found that we had named a file incorrectly. During experimentation, we had a typo; BAY 4 should be `WCC2EE134983` but that's not what we saw when we checked our work. Not the end of the world. We can move the .eli metadata file, and we can use gpart[134] modify to change the label.

```
 gpart modify -i 1 \
-l ZFS-BLONDIE-4-SWAP-WCC2EE134983 mfid4
```
[135]

```
gpart modify -i 2 \
-l ZFS-BLONDIE-4-FILES-WCC2EE134983 mfid4
```
[136]

`gpart show`[137] and `geli status`[138] should show our drives as we expect them to be.

At this point, all of the drives should be partitioned and decrypted and attached. We can build a zpool for our ZFS file system. We can attach that swap drive for the benefit of our operating system right now.

To Use the Swap Disk

`swapon mfid0p1.eli`[139] `swapinfo -k`[140]

64G turned out to be the maximum capacity. We will need to fix up the swap better. We can fit 6 x 64G (384G) of swap partitions in our planned space. So, let's modify and add.

`gpart resize -i 1 -s 64G mfid0`[141]

We would need to add, to init and to attach five more swap partitions. We also have to restore mfid0 because there's a problem with reading the metadata.

`gpart recover mfid0`[142]
`gpart status mfid0`[143]
`geli attach mfid0p1`[144]

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-B-WCC2ERZ56512 mfid0
```
[145]

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-C-WCC2ERZ56512 mfid0
```
[146]

```
gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-D-WCC2ERZ56512 mfid0[147]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-E-WCC2ERZ56512 mfid0[148]

gpart add -s 64G -t freebsd-swap \
-l ZFS-BLONDIE-0-SWAP-F-WCC2ERZ56512 mfid0[149]
```

Then we need to establish encryption for all of those.

```
geli init -l 256 mfid0p2[150]
geli attach mfid0p2[151]
geli init -l 256 mfid0p3[152]
geli attach mfid0p3[153]
geli init -l 256 mfid0p4[154]
geli attach mfid0p4[155]
geli init -l 256 mfid0p5[156]
geli attach mfid0p5[157]
geli init -l 256 mfid0p6[158]
geli attach mfid0p6[159]
```

Using `swapinfo`[160], `swapon`[161] and `swapoff`[162], we were able to remove some of the other swap partitions and add on these. Did see a kernel limit on swap. Total configured swap exceeded kern.maxswzone (98087976 pages). We could tune the kernel, or we could reduce swap. For now, we reduce swap from 384GB to 324GB.

Simliarly, we could add on the other swap partitions, although in our current use, we probably won't need them. They would serve us better in cases where we were doing more intensive work, or working on another machine. We can mount them now, anyway, to see that it can be done.

To add on all the other swap partitions:

```
swapon /dev/mfid4p1.eli[163]
swapon /dev/mfid5p1.eli[164]
swapon /dev/mfid6p1.eli[165]
swapon /dev/mfid7p1.eli[166]
swapinfo -h[167]
```

Create the zpool for BLONDIE

We'll now create a mountpoint on the tmpfs, create the zpool, and then add in the cache and SLOG Mirrored disks.

Create a mountpoint

Create a mountpoint so that we will be able to mount the zpool without errors as we create it.
`mkdir /tmp/BLONDIE`[168]

2.4 Analysis

2.4.1 Quality

2.4.2 Tests

2.5 Discussion

2.6 Chapter References

2.6.1 Books

For our ZFS planning, we read: Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

2.6.2 MAN Pages

Online MAN pages were available at: man.freebsd.org with an interface that allows for keyword based searches.

"bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"camcontrol." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=camcontrol&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"cd." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cd&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"cp." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cp&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.

"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-R

ELEASE&arch=default&format=html. Accessed 10 June 2026.

"ls." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ls&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"mt." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mt&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"newfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=newfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"rm." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=rm&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"ssh-Keygen." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"su." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=su&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"swapon." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"umount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=umount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

"zpool." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

2.6.3 Websites

When planning and exploring our use of swap, we consulted: https://www.cyberciti.biz/faq/create-a-freebsd-swap-file/.

Endnotes

[1]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[2]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[3]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[4]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[5]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[6]Vppalapati, Mallikarjun. "Immutability as an Operational Constraint rather than a Security Feature." International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[7]Wolford, Ben. "Everything You Need to Know About the 'Right to Be Forgotten.'" GDPR.Eu, 5 Nov. 2018, https://gdpr.eu/right-to-be-forgotten/. Accessed 12 July 2026.

[8]GDPR. "Art. 17 GDPR - Right to Erasure ('Right to Be Forgotten') - GDPR.Eu." GDPR.Eu, 14 Nov. 2018, https://gdpr.eu/article-17-right-to-be-forgotten/. Accessed 12 July 2026.

[9]"Recital 65 - Right of Rectification and Erasure." GDPR.Eu, 14 Nov. 2018, https://gdpr.eu/recital-65-right-of-rectification-and-erasure/. Accessed 12 July 2026.

[10]"Recital 66 - Right to Be Forgotten." GDPR.Eu, 14 Nov. 2018, https://gdpr.eu/recital-66-right-to-be-forgotten/. Accessed 12 July 2026.

[11]Chandramouli, Ramaswamy. Guidelines for Media Sanitization. 2025, nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-88r2.pdf, https://doi.org/10.6028/nist.sp.800-88r2.

[12]"Blondie | the Official Website of Blondie, Featuring Tour Dates, Presale Ticketing, News, the Official Store and More." Blondie.net, 2026, blondie.net/. Accessed 10 June 2026.

[13]"LTO Ultrium 3: Product Views | Fujifilm [United States]." Fujifilm.com, 2026, www.fujifilm.com/us/en/business/data-storage/data-storage-media/lto-ultrium-3/product-views. Accessed 10 June 2026.

[14]"WD VelociRaptor ® SATA Hard Drives" https://theretroweb.com/storage/documentation/wd6000-velociraptor-68407cb5c0c96653507203.pdf

[15]Intel® Xeon® Processor 5500 Series an Intelligent Approach to IT Challenges. www.intel.com/content/dam/doc/product-brief/xeon-5500-server-brief.pdf.

[16]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[17]This pattern of coordinating host computer, bay position, and drive serial number was recommended by Lucas and Jude. Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[18]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[19]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[20]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[21]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[22]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[23]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[24]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[25]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[26]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[27]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[28]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[29]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[30]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[31]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[32]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[33]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[34]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[35]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[36]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[37]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[38]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[39]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[40]"Chapter 6 Managing ZFS File Systems (Solaris ZFS Administration Guide)." Docs.oracle.com, docs.oracle.com/cd/E19120-01/open.solaris/817-2271/6mhupg6m3/index.html.

[41]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[42]zfs set shown in: "Zfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.0-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[43]Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[44]Lucas and Jude provide a detailed breakdown of different RAID and RAID-Z combinations with differing numbers of drives. Their analysis and provided charts were our cheif reason for settling in on RAIDZ2. Lucas, Michael W, and Allan Jude. FreeBSD Mastery: ZFS. Tilted Windmill Press, 21 May 2015.

[45] "bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

[46]Ibid.
[47]"bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html
[48]"bsdinstall." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=bsdinstall&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html
[49]"ssh-Keygen." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[50]"camcontrol." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=camcontrol&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[51]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026
[52]Ibid.
[53]"newfs." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=newfs&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[54]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[55]"Mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[56]"cd." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cd&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[57]"umount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=umount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[58]"rm." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=rm&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[59]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[60]"Mount." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[61]"ls." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ls&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[62]"Mkdir." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[63]"ssh-Keygen." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=ssh-keygen&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[64]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[65]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[66]Ibid.

[67]Ibid.
[68]Ibid.
[69]Ibid.
[70]Ibid.
[71]Ibid.
[72]Ibid.
[73]“zpool.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[74]Ibid.
[75]Ibid.
[76]“Mkdir.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[77]“mount.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mount&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[78]“gpart(8).” Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[79]Ibid.
[80]Ibid.
[81]Ibid.
[82]Ibid.
[83]Ibid.
[84]Ibid.
[85]Ibid.
[86]Ibid.
[87]Ibid.
[88]Ibid.
[89]Ibid.
[90]Ibid.
[91]Ibid.
[92]Ibid.
[93]Ibid.
[94]Ibid.
[95]Ibid.
[96]Ibid.
[97]Ibid.
[98]“Geli(8).” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
[99]Ibid.
[100]Ibid.
[101]Ibid.
[102]Ibid.
[103]Ibid.
[104]Ibid.
[105]Ibid.
[106]Ibid.
[107]Ibid.
[108]Ibid.
[109]Ibid.
[110]Ibid.
[111]Ibid.
[112]Ibid.
[113]Ibid.
[114]Ibid.

[115]Ibid.
[116]Ibid.
[117]Ibid.
[118]Ibid.
[119]Ibid.
[120]Ibid.
[121]Ibid.
[122]"cp." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=cp&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[123]Ibid.
[124]Ibid.
[125]Ibid.
[126]Ibid.
[127]Ibid.
[128]Ibid.
[129]Ibid.
[130]Ibid.
[131]Ibid.
[132]Ibid.
[133]Ibid.
[134]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[135]Ibid.
[136]Ibid.
[137]Ibid.
[138]"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
[139]"swapon." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[140]Ibid.
[141]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[142]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[143]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[144]"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
[145]"gpart(8)." Freebsd.org, 2026, man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=8&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[146]Ibid.
[147]Ibid.
[148]Ibid.
[149]Ibid.
[150]"geli(8)." Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=geli&sektion=8&manpath=FreeBSD+10.3-RELEASE. Accessed 10 June 2026.
[151]Ibid.
[152]Ibid.
[153]Ibid.
[154]Ibid.

[155]Ibid.
[156]Ibid.
[157]Ibid.
[158]Ibid.
[159]Ibid.
[160]“swapon.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[161]Ibid.
[162]Ibid.
[163]“swapon.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=swapon&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.
[164]Ibid.
[165]Ibid.
[166]Ibid.
[167]Ibid.
[168]“mkdir.” Freebsd.org, 2025, man.freebsd.org/cgi/man.cgi?query=mkdir&apropos=0&sektion=0&manpath=FreeBSD+10.3-RELEASE&arch=default&format=html. Accessed 10 June 2026.

3 Login Procedures for Disconnected Systems

This outline will help us to log into a laptop that is set up to be a disconnected, air-grapped machine. We cover what items are needed, and how to attach and to decrypt the disks. The operating system is provided through immutable, read-only, optical media. Some key values were encoded onto magstripe cards. Others were recorded in a paper notebook. The machine now has a refined set of startup procedures that are incompatible with normal laptop use. A general outline of the machine's login procedures are listed here.

3.1 Introduction

We wanted to establish an air-gapped machine for off-network use. To protect critical data and programs, we wanted to have a recoverable immutable operating system with encrypted storage. To inhibit networking, we permanently disabled the wired and wireless systems. We prepared on-machine and peripherial encrypted storage.

Our intention is to use the laptop as a general-purpose air-grapped machine. We want to be able to generate keys and certificates, to access and to edit secrets, and to develop small scripts offline.

The login procedure for this disconnected laptop is similar to a procedure used for disconnected rack servers. Due to the portability of the laptop, and the frequent use of some of the authentication challenges, the laptop has some magstripe additions.

3.2 Methodology

Booting this machine requires a few phases:

1. Physical access.
2. POST and BIOS.
3. Disk discovery and attachment.
4. Disk decryption.
5. Dataset attachment.

Powering off the machine requires a similar reversed sequence:

1. Dataset detachment.
2. Encrypted partition detachment.
3. Disk detachment.
4. Poweroff.
5. Physical security of related parts.

Of these, discovering and attaching the disks for use may be the biggest stumbling block for the uninitiated. Manually decrypting the disks is a close second. Attaching a dataset is very similar to normal ZFS use. Most of these tasks are done automatically by operating systems. In this system, the steps have been disconnected. This allows us to swap out storage and provide a measure of procedural security to the system.

3.3 Results

3.3.1 Required Or Specific Equipment

We used a Dell XFR laptop that was purchased from a police auction. We replaced the main hard drive. We disassembled the machine and prepared it for air-grapped use. The Ethernet connection jack was jammed with epoxy putty. The camera and microphone assemblies were removed. The wifi card and antennas were removed. A privacy screen of polarized vinyl was installed over the display. USB ports were temporarily blocked with plugs that can be removed with a specialized tool. The laptop has been fitted with a bluetooth tracking device and a tinted luggage tag.

Some key values are stored on encrypted USB drives that are equipped with a read-only physical switch. If this switch is not moved into the "write" position, then the disk will not successfully decrypt. The drives were encrypted with Camelia using geli in a ZFS partition set up with gpart. The USB drives are labelled with color-coded tags and inventory stickers to identify the classification of data held on them and the drives' individuality.

For convenience, some key values have been written to magstripe cards. This is nearly equivalent to writing the passwords down in plain text. If the card reader is connected, when the card is swiped, it will reveal its values wherever the cursor is available. Inputting text from a magstripe card is not much different from using a keyboard. To get the magstripe to read properly, character sets and deliminating characters have to be used to construct the password values. The specific magstripe cards and a magstripe reader are needed if the passphrases are not input manually.

For paper backups, some key values have been written in UV-readable "invisible" ink in a notebook. To use this notebook, a UV penlight is kept on hand. To update that notebook, UV ink is loaded into refillable cartridges of a fountian pen.

3.3.2 Code Notes for Power Up

1. Assemble the needed components:
    - The laptop
    - Power for the laptop
    - USB software-encrypted storage with read-only disk
    - USB hardware encrypted disk (if needed) with software encrypted storage dataset
    - Codebook with UV lights and inks (if needed)
    - Magstripe reader
    - Any other mutlifactor devices needed for specific systems (biometric readers, TOTP/HOTP authentication devices, certificates, etc.).
2. Attach peripherals and power up the laptop.
   USB temporary plugs may need to be removed. Connect the magstripe reader, if needed. Be prepared to connect any USB drives that get used.

3. BIOS password.
   BIOS settings are usually tuned to have the boot order boot the system from optical disk. If needed, use the function key to select which disk is needed for a one-time boot. There will only be a few seconds in the startup to to press the correct key.
4. Use a recent FreeBSD bootonly.iso disk on optical media.
   This will only have the most primitive collections of programs and will not be sufficient for certificate generation due to the libraries that are provided. The operator should be able to hear the disk spin up.
5. At the menu, select LiveCD to get a root prompt in terminal.
6. Attach and discover disks.
   If the disks are hardware encrypted, then provide the PIN according to the manufacturer's directions. Enter the PIN carefully, as there may be administrative reset or disk-wiping functions available. With the disk hardware decrypted and attached, a message should be visible on the console describing the detection of the device.
   `geom disk list` - should show the disks recognized by the system.
   `gpart show` - should provide an indicator of partition numbers
7. Attach and decrypt partitions `geli attach [DISK_NAMEpPARTITION_NUMB`
   If a passphrase was recorded on magstripe, this is the point where the dedicated card will need to be swiped. If the card does not work, then this is the point where the paper UV ink backup notes may be used.
8. Discover and attach zpools and zfs datasets. `zpool import POOL_NAME_ON_AT`
   `zpool import -af` to forcefully attach all available zpools `zpool status`
   If a MAC kerenel mod like mac_biba is used on the file system, then the system may be a UFS file system on a ZFS Volume. These are often mounted with mount from a /dev device node or a GEOM label. [Advanced ZFS]
9. Observe datasets. Notes should tell us where to go to see where the datasets have automatically mounted. Or, we can use zfs commands to set a mountpoint to a desired location. In the read-only LiveCD media, our most likely attachment point may be /tmp.
10. Use the laptop to interact with files on the storage.
11. Decrypt individual files with openssl.

3.3.3 Code Notes for Poweroff

The poweroff procedure is similar to a reverse of the poweron, but there are fewer steps. If the ZFS elements are not properly exported, then future imports may require a `-f` force attribute on the commands. Notice also if there are automount directories associated with a ZFS dataset, then these will not be destroyed upon dismount. Instead, they will appear empty. Hardware drives will generally re-encrypt once disconnected from the machine.

With a LiveCD or El Torrito in FreeBSD, it is very common for the system to reboot after a shutdown command. Watch your system carefully to avoid spending laptop battery power after giving a poweroff command. Notice also that the laptop will not go into a suspend or hibernate mode if the laptop display is closed. Those are advanced gui and system functions which will not be covered in El Torrito.

1. Dispose of decrypted copies of any files that are in the plain. Ensure that an encrypted copy continues to live on the storage drives. Re-encrypt if necessary.

2. Dismount media. `zfs export [DATASET_NAME] geli detatch [DRIVE.eli] geom disk list`

In order to dismount the media, do not have a terminal command line open on any directories associated with that media. To confirm a directory is empty after dismount, use ls.

3.4 Analysis

3.4.1 Quality

We have some fragile areas of the system.

We can swipe a card with the password in an environment where reading a password from a book might be uncomfortable. The speed of the magstripe's use can be a blessing during development and testing or other times when the challenges may be frequently encountered. However, the password will need to be stored in plain text on the magnetic stripe. This means that magstripe cards used for passwords will need physical access control safeguards.

BIOS is used instead of UEFI to keep the boot process simple and uniform with other machines.

USB detachable storage has been a well-known security weak point for over a decade. We have chosen USB port blockers; but, those frequently come with proprietary plastic keys to insert or extract them. It's obvious that those port blockers are an inconveniece device; they don't provide full access controls; they're usefulness is limited to slowing down the surrepetitious attachment of USB devices.

3.4.2 Tests

Disk metadata backups. geli is frequently used in automatic installations of FreeBSD systems using bsdinstall. However, those automatic installs don't prompt an admin for saving a copy of the disk metadata. If that metadata file is not saved someplace where its contents can be carefully preserved, then any difficulty with decrypting the disk may be scuttled. The recovery commands for geli will often require or prompt for a path to the metadata file.

Not only should the metadata be saved, but use of it should be periodically rehearsed. Since it is needed for disk recovery, and period since the last recovery rehearsal should be considered a time of unconfirmed disk storage. That data is in jeopardy of being lost to a disk decryption problem. Rehearse the recovery of disks with the metadata files. Without them, the only option in the event of a disk decryption problem will be to start over with a geli init to basically reformat the encrypted drive partition.

3.5 Discussion

3.5.1 Summary of Usefulness

We believe that this disposition of resources will position us well to intervene on attempts at root-level persistence. So many machines these days are configured to boot into an operating system on power-up that a disconnected system might not be anticipated by an attacker.

The disconnected system has also been useful for swapping out storage drives on a machine. Many machines are being built and sold with on-motherboard RAM and immovable SSDs. While this supports thinner designes for single-board laptops, it doesn't suit our experimentation methods on salvaged equipment.

3.5.2 Lessons Learned

This type of system requires some practice for routine use. When using disconnected systems or live-disc systems, the configurations may not persist between boots. A lack of rehearsals or familiarity with the boot and power-up procedure may cause delays as the operator teaches themselves the pattern or troubleshoot omitted steps.

3.6 References

3.6.1 Books

Lucas, Michael W. and Jude, Allan. FreeBSD Mastery: ZFS. 2015. ISBN-13: 978-1-64235-000-5.

We found this book to be an excellent guide to ZFS, overall. We also adopted naming suggestions that aligned exterior drive serial numbers with disk labelling. Chapter 2 featured a helpful guide on planning RAIDZ configurations with notes on efficiency. Advice on snapshots, cloning, and block devices for non-ZFS file systesms were all helpful.

3.6.2 MAN pages

`openssl enc` FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=enc&apropos=0&sektion=1&manpath=freebsd-ports&format=html

`zfs` Freebsd MAN pages, https://man.freebsd.org/cgi/man.cgi?query=zfs&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

`zpool` FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=zpool&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

`geom` FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=geom&apropos=0&sektion=0&manpath=FreeBSD+1

4.1-RELEASE+and+Ports&arch=default&format=html

geli FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=geli&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

gpart FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=gpart&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

dd FreeBSD MAN pages, https://man.freebsd.org/cgi/man.cgi?query=dd&apropos=0&sektion=0&manpath=FreeBSD+14.1-RELEASE+and+Ports&arch=default&format=html

3.6.3 URLs

FreeBSD Handbook, ZFS, https://docs.freebsd.org/en/books/handbook/zfs/

FreeBSD Handbook, Storage, https://docs.freebsd.org/en/books/handbook/disks/

Text with[1] inline references. More text[2]

Endnotes

[1]First note.
[2]Second note.

4 Basic Tape Operations with FreeBSD

We explored the planning and use of LTO tape as a resource for storing backups. We describe common commands, patterns of MT and TAR commands. We recommend some techniques for testing, rehearsing, and using tape archives.

4.1 Introduction

We wanted to see if it would be possible for us to make use of an LTO tape machine that was on one of our salvaged computers. We'd often heard of the performance of using tape for backups. We'd seen some hints about the bravery needed for working with tape in modifying old mainfRAMe progRAMs. We rarely saw any strong tutorials or comprehensive directions for tape operations.

The average person has almost never used tapes as a part of personal computing these days. Businesses which do use tapes often do so as part of larger, commercial data center operations. For many of us, the closest we've come to tape operations was with using cassette tapes for storage on personal machines in the 1980s. LTO tapes are often praised but infrequently used. We wanted to break through this mystery.

4.2 Methodology

Our plan is to write a file to tape, eject the tape, and be able to read the file back from the tape again. Once there, we will want to do a simple sequence of file writes and reads without damaging the files on tape.

4.3 Results

We used a Dell T610 tower computer with an LTO3 single deck tape machine on board. We have some LTO3 tapes that can be used with it. Our tape drivers and common commands are built into FreeBSD.

Warning: We want to use the nsa driver. There is an sa driver, but it has some limitations. When we tried using sa, we were not able to properly advance down the tape through multiple file markers. Use nsa for the the driver.

4.3.1 Code Notes

1. Place files in a directory that you control. Isolating the files to be copied to tape is safer than copying them directly from where they are.

2. `mt -f /dev/nsa0 load`

This will load the tape onto the machine.

3. `mt -f /dev/nas0 rewind`

This will spool the tape to the beginning.

4. `mt -f /dev/nsa0 status`
To check to see where we are on the tape. Assuming this is the first use of the tape, we could start writing immediately.

5. If there are files on the tape, we need to either scan ahead and log the files, or we need to consult our notes to find where on the tape we should go. We could easily overwrite a file and damage the sequence. Use the write persistent notch to protect the tape when possible.

6. `mt -f /dev/nsa0 com or eod`
To go to the end of the records.

7. `tar cvf /dev/nsa0 [FILE_TO_ARCHIVE]`
Once at the proper spot on the tape, give the command to archive the files.

8. `mt -f /dev/nsa0 weof1`
Write an end-of-file marker.

9. `tar -t`
To see the archive on the tape.
`tar xf /dev/nsa0`
To get the archive from the tape.

10. Write notes. Log and record what was put on the tape.

11. For the next record, just rewind and advance through the markers to get to the next blank space for writing a record.

Sample Files: test.txt

```
The quick brown fox jumped over the lazy dogs.
```

Sample Files: test2.txt

```
The other dog is still lazy, too.
```

Sample Files: test3.txt

```
The fox is just really fast.
```

Sample Files: test4.txt

```
Yet some more records.
```

Sample Files: test5.txt

```
And another record.
```

Sample Script: scriptedDemo.sh

```
#!/usr/bin/env sh
clear
echo "scriptedDemo"
echo "Press ENTER to continue."
```

```
read INPUT
## Check tape
echo "Check if tape is staged in machine."
echo "Press ENTER to continue."
read INPUT
clear
## Load the tape into the machine
echo "Here we go."
echo
echo "Loading tape into machine."
echo "mt -f /dev/nsa0 load"
mt -f /dev/nsa0 load
## Rewind tape
echo "Rewinding tape"
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "Press ENTER to continue."
read INPUT
clear
echo "Reading status from tape."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for file 0 on the tape."
echo "We should be at the beginning."
echo "So, no mt commands this time."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for file 1 on tape."
echo "Rewind to the beginning."
mt -f /dev/nsa0 rewind
echo "Go forward space count 1 file."
echo "mt -f /dev/nsa0 fsf 1"
mt -f /dev/nsa0 fsf 1
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
```

```
clear
echo "Looking for file 2 on tape."
echo "Back up to 0 and go forward space count 2 files."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 fsf 2"
mt -f /dev/nsa0 fsf 2
echo "Show where we are with status."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "Look in that spot with tar."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
clear
echo "Looking for third archive on tape."
echo "Back up to 0 and go forward space count 3 files."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 fsf 3"
mt -f /dev/nsa0 fsf 3
echo "See where we are with status."
echo "mt -f /dev/nsa0 status"
mt -f /dev/nsa0 status
echo
echo "See what is here with tar."
echo "tar t -f /dev/nsa0"
tar t -f /dev/nsa0
echo "Press ENTER to continue."
read INPUT
clear
echo "Take the tape offline."
echo "Rewind tape."
echo "mt -f /dev/nsa0 rewind"
mt -f /dev/nsa0 rewind
echo "mt -f /dev/nsa0 offline"
mt -f /dev/nsa0 offline
exit 0
```

4.4 Analysis

4.4.1 Quality

1. We found there were significant weaknesses in tape sequencing. Unlike random access memory, overwriting can damage end of file markers which are only navigational delimiters readily usable with the tape. Simple practice showed it was very easy to accidentally overwrite a tape marker. All subsequent files on the tape might be inaccessible as a result.

2. Logging. Since there may be many files on these long tapes, logging support and file navigation are essential. The usual lapses in notation can make frequent use of the tapes more hazardous with each use.

3. Security. Unlike geli-encrypted hard drives, the main file protection scheme is through whatever encryption we apply to the files themselves. Tar has some compression algorithms, but encrypting files is another step altogether. Since the files may be in storage for a long time, having the backups encrypted on tape can be a critical step. This encryption would need to be applied before IV.B.7. tar or archiving the files to tape.

4.4.2 Tests

Writing to tape should be rehearsed. Checking status and reading files to confirm our location on the tape is a good idea.

Any write operation should be couched on a good, rehearsed read script. Isolate the files to be written to their own directory. Add an encryption step if necessary. Rehearse the tar tape archive commands. Compartmentalize the steps of the tape operations plan to reduce catastrophic failures and a loss of data.

4.5 Discussion

Tape operations have been long recommended as the preferred form of backup. Offsite backup offers strong advantages to an on-premesis system operator. We can't count the number of times we've seen backups recommended; yet we find very few practical tutorials or directions.

Tape backups and hard drive backups might be easily and securely routed to an off site location through the US mail. They could be mailed to yourself at a personal mailbox, remote office, P.O. Box, or to a trusted person. Notice that any person or entity entrusted to possess the data may also be at risk or some sort of liability.

4.6 References

A. Books – none. 2023 versions of the FreeBSD handbook don't cover tape operations.

B. man pages – `mt`, `tar` man pages.

C. URLs – FreeBSD man pages for above. `man.freebsd.org/cgi/man.cgi?` query mt tar

Text with[1] inline references. More text[2]

Endnotes

[1]First note.

[2]Second note.

Glossary

amnesiac The Oxford English Dictionary (OED) described amnesiac as (1913-), "One who is afflicted with amnesia." [1] In this context, an amnesiac system is one that will forget recent changes; usually, the system has an immutable operating system as a defense against attacks.

BSD Berkeley Software Distribution, a Unix-derived operating system and its variants (FreeBSD, OpenBSD, NetBSD).

cache The OED described it as, "3. 1968– Computing. A small high-speed memory in some computers into which are placed the most frequently accessed contents of the slower main memory or secondary storage. Also cache memory." [2]

CCSP Certified Cloud Security Professional

CISSP Certified Information Systems Security Professional

coding The OED described coding as (1946-), "3. Computing. The conversion of information into code usable by a computer or other machine; the writing or editing of code for a computer program, application, etc. Also: computer code." [3]

configuration The OED described configuration as (1962), "8. Computing. The way the constituent parts of a computer system are chosen or interconnected in order to suit it for a particular task or use; the units or devices required for this." [4]

CSSLP Certified Secure Software Lifecycle Professional

dialog OED(1) says, "4.b. 1984– Chiefly in form dialog. = dialogue box n." [5] OED(2) says, "Computing. 1984- A window in a graphical user interface which asks the user for input in response to some given information, such as a choice of command in answer to a question." [6]

encrypt The OED described it as, "transitive. To convert (data, a message, etc.) into cipher or code, esp. in order to prevent unauthorized access; to conceal in something by this means." [7]

EU European Union

GDPR General Data Protection Regulation

GPU Graphics Processing Unit

hard immutability A system whose unchangeable nature is so constrained by technical mechanisms that alteration is practically infeasible. [8]

ICC Internet Crime Center

immutable system The OED described immuatable as (1621), "1.b. technical. Not subject to variation in different cases; invariable: used e.g. of markings which are the same in all the individuals of a species." [9] An immutable system is one that cannot be changed. In these computing systems, the intent of using an immutable system is to allow an operator to restore a system to its original state by power cycling it.

jail A FreeBSD OS-level virtualisation mechanism that partitions the operating system into isolated environments.

memory stick The OED described it as, "(Originally) a type of memory card for use in digital cameras and other devices; (now chiefly) a USB flash drive, esp. a small one in the shape of a long and relatively flat rectangular prism. A proprietary name." [10]

metadata The OED described it as, "Data whose purpose is to describe and give information about other data." [11]

mirror OED described it as (1993-), "4. transitive. Computing. To write (data) on to two separate devices (esp. two hard disks) simultaneously, to protect against the possible failure of one; to create (a duplicate disk) in this manner. Also: to copy (a website) on to a different server (cf. mirror site n.)." [12]

motherboard The OED described it as (1965-), "Electronics. A printed circuit board on which the principal components of a microcomputer or other electronic device are mounted, and to which other boards (daughterboards) may be connected." [13]

NIC Network Interface Card

NIST National Institute of Standards and Technology

OED Oxford English Dictionary

operating system The OED described it as (1961-), "Computing. The low-level (low-level adj. 4b) software that supports a computer's basic functions, such as scheduling tasks, controlling peripherals, and allocating storage."[14]

operational immutability A system that restricts changes by requiring a specific technology. [15]

OS Operating System

PII Personally Identifiable Information

pipe The OED had a couple relevant descriptions for it. "III.26.b. 1979- A physical or notional data channel or route." [16] Also, "III.26.c. 1982– A temporary connection between two processes or commands, so that the output from one command becomes the input for the next." [17]

Poudriere Named after the French word for "powderkeg," Poudriere is a package building program that will custom-compile programs from ports and configuration files. A FreeBSD operator can then use their own tailored programs built to specifications while obtaining updates from the ports tree.

RAID The OED described it as (1988-), "Computing. Redundant array of inexpensive disks (or drives), a data storage system which combines multiple hard drives to form a system functioning as single drive." [18]

RAM Random Access Memory
RtbF Right to be Forgotten

SAS Serial Attached SCSI
SATA Serial AT Attachment
shell The OED described it as (1974-), ”b. In some operating systems (originally Multics and Unix): a program that translates commands keyed by the user into commands that the operating system can act on, thereby providing a high-level interface with the user. Also (more fully shell window): a window in which such commands can be invoked.” [19]
soft immutability A system that limits changeability through technically proscribed permissions. [20]
SSD Solid State Drive
SSH Secure Shell

thumbdrive OED described it as, ”a. A mechanism that is operated by the thumb; b. Computing (a proprietary name for) a USB flash drive, esp. a small one in the shape of a long and relatively flat rectangular prism; cf. memory stick n.” [21]

US United States

VNET Virtual Network

Endnotes

[1] “Amnesiac, N.” Oxford English Dictionary, Oxford UP, December 2024, https://doi.org/10.1093/OED/9207200444.

[2] “Cache, N. (2), Sense 3.” Oxford English Dictionary, Oxford UP, September 2025, https://doi.org/10.1093/OED/1117661132.

[3] “Coding, N., Sense 3.” Oxford English Dictionary, Oxford UP, June 2025, https://doi.org/10.1093/OED/4701717734

[4] “Configuration, N., Sense 8.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/1065280211

[5]“Dialogue | Dialog, N., Sense 4.b.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/4999880636.

[6] “Dialogue Box | Dialog Box, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/3367209004.

[7] “Encrypt, V.” Oxford English Dictionary, Oxford UP, December 2025, https://doi.org/10.1093/OED/1124748007.

[8]Vppalapati, Mallikarjun. “Immutability as an Operational Constraint rather than a Security Feature.” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[9]“Immutable, Adj., Sense 1.b.” Oxford English Dictionary, Oxford UP, December 2025, https://doi.org/10.1093/OED/1176087402.

[10] “Memory Stick, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/1531282927.

[11] “Metadata, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/7968104326.

[12] “Mirror, V., Sense 4.” Oxford English Dictionary, Oxford UP, December 2025, https://doi.org/10.1093/OED/8366938443.

[13]“Motherboard, N.” Oxford English Dictionary, Oxford UP, July 2023, https://doi.org/10.1093/OED/1154882806.

[14] “Operating System, N.” Oxford English Dictionary, Oxford UP, July 2023, https://doi.org/10.1093/OED/4828064025.

[15]Vppalapati, Mallikarjun. “Immutability as an Operational Constraint rather than a Security Feature.” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[16] “Pipe, N. (1), Sense III.26.b.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/5943533372.

[17] “Pipe, N. (1), Sense III.26.c.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/8044200378.

[18] “RAID, N.” Oxford English Dictionary, Oxford UP, June 2026, https://doi.org/10.1093/OED/4981410541.

[19] “Shell, N., Additional sense.” Oxford English Dictionary, Oxford UP, March 2026, https://doi.org/10.1093/OED/4563192747.

[20]Vppalapati, Mallikarjun. “Immutability as an Operational Constraint rather than a Security Feature.” International Journal of Artificial Intelligence, Data Science, and Machine Learning, vol. 3, no. 2, 30 June 2022, pp. 176–184, https://doi.org/10.63282/3050-9262.ijaidsml-v3i2p119. Accessed 21 June 2026.

[21] “Thumb Drive, N.” Oxford English Dictionary, Oxford UP, March 2026, https://doi.org/10.1093/OED/9979951645.

Index